DAY 14 | HANDS-ON ACTIVITIES

Data Pipelines and Spark Intro: Problem Statements

Seven problems building a PySpark pipeline. In Colab, Spark is ready to use. In VS Code, install Java 17 and pyspark first.

Prepared by **Ahmed** | linkedin.com/in/ahmed-al-imaad

How to work: read the problem, write the steps in plain English first, then write the code. Test with the sample run, then try your own inputs, including wrong ones.

Problem 1: Hello Spark

Level: Warm-up | **Concepts:** SparkSession, createDataFrame, show

Start Spark, create a DataFrame of 5 cities with population and country, print the schema, show the rows and count them.

Sample run / expected result

```
root
 |-- city: string
 |-- population: long
...
Count: 5
```

Hints

- spark.createDataFrame(list_of_tuples, ["city", "population", "country"])

Problem 2: Filter and Derive

Level: Easy | **Concepts:** filter, withColumn, when

From the orders data, keep amount > 0, trim city names, and add a size column: large (>= 1000), medium (>= 300) or small.

Sample run / expected result

```
|order_id|city     |amount|size  |
|1002    |Tokyo    |1200.0|large |
```

Hints

- Chain F.when(...).when(...).otherwise(...).

Problem 3: Lazy or Not?

Level: Easy | **Concepts:** transformations vs actions

Write 4 transformations without any action and time them, then call count() and time it. Explain the difference in two sentences.

Sample run / expected result

```
Transformations: 0.01 s
count(): 1.8 s
```

Hints

- import time; time.perf_counter()

Problem 4: Daily Revenue

Level: Medium | **Concepts:** groupBy, agg, orderBy

Compute orders, revenue and average order per day and city, sorted by revenue descending.

Sample run / expected result

```
|order_date|city     |orders|revenue|avg_order|
```

Hints

- F.count, F.sum, F.round(F.avg(...), 2)

Problem 5: Join Detective

Level: Medium | **Concepts:** inner, left, left_anti

Join orders with customers using inner and left joins and compare row counts. Use left_anti to list orders whose customer is missing.

Sample run / expected result

```
orders: 10 | inner: 8 | left: 10
Orders with unknown customer: 1009, 1010
```

Hints

- Count before and after every join.

Problem 6: Medallion Pipeline

Level: Challenging | **Concepts:** bronze, silver, gold, Parquet

Write raw data as bronze Parquet, clean it into silver, join and aggregate into gold daily revenue by city, partitioned by date. Re-run and show counts are unchanged.

Sample run / expected result

```
bronze: 12 | silver: 9 | gold: 7
Re-run: same counts
```

Hints

- mode("overwrite") makes re-runs idempotent.

Problem 7: Top City per Day

Level: Stretch | **Concepts:** window functions

Using the gold table, find the city with the highest revenue for each day using a window function and rank().

Sample run / expected result

```
2026-11-12  Tokyo  4500.0
2026-11-13  Bengaluru  3100.0
```

Hints

- Window.partitionBy("order_date").orderBy(F.desc("revenue"))

Ganbatte (頑張って): do your best